TECHNICAL DOCUMENTATION

Row-Level Security (RLS)

Comprehensive Study Notes: Definition, Core Concepts, Implementations & Best Practices

1. Executive Summary & Definition

Formal Definition

Row-Level Security (RLS) is a fine-grained database access control mechanism that restricts data access at the row level based on the characteristics of the user executing a query. Instead of granting or denying access to an entire table or column, RLS dynamically filters records, ensuring users only see or modify data for which they have explicit authorization.

In traditional security models, permissions are coarse-grained (e.g., full `SELECT` access to a table). RLS introduces *context-aware filtering* at the storage engine level. When a user runs ***SELECT * FROM orders***, the database engine automatically injects background predicates (e.g., ***WHERE tenant_id = CURRENT_USER()***) before evaluation.

2. Fundamental Architecture & Core Concepts

1. Security Predicates

Filter functions or inline logical expressions (e.g., `WHERE department_id = user_dept()`) evaluated silently for every row touch.

2. Policy Scope

Rules applied specifically per command execution: `SELECT` (Read), `INSERT` (Create), `UPDATE` (Modify), or `DELETE` (Remove).

3. Session / User Context

Runtime environment state storing identity claims, user roles, or tenant parameters (e.g., `SESSION_CONTEXT`, `current_setting()`).

4. Permissive vs. Restrictive

Permissive: Combined via `OR` logic (grant access if any match).

Restrictive: Combined via `AND` logic (must satisfy all policies).

3. Static vs. Dynamic RLS

DIMENSION	STATIC ROW-LEVEL SECURITY	DYNAMIC ROW-LEVEL SECURITY
Mechanism	Hardcoded values assigned directly to user roles/groups in the policy.	Evaluated dynamically at query execution using session contexts or lookup tables.
Maintainability	Low. Requires policy modification whenever users or roles change.	High. User-to-data mappings are stored in metadata tables without schema alterations.
Scalability	Limited to environments with static, small user groups.	Scales seamlessly to multi-tenant architectures with millions of users.

DIMENSION	STATIC ROW-LEVEL SECURITY	DYNAMIC ROW-LEVEL SECURITY
Typical Use Case	Regional managers (e.g., Europe vs. North America teams).	SaaS Multi-tenancy, User-specific personal data, dynamic organization hierarchies.

4. Implementation Examples (PostgreSQL & Power BI / DAX)

A. PostgreSQL Row-Level Security

```
-- 1. Enable RLS on target table
ALTER TABLE sales_data ENABLE ROW LEVEL SECURITY;

-- 2. Create a dynamic policy bound to session variables
CREATE POLICY regional_sales_policy ON sales_data
    FOR ALL
    TO application_role
    USING (region = current_setting('app.current_region'))
    WITH CHECK (region = current_setting('app.current_region'));

-- 3. Application sets session variables prior to execution
SET LOCAL app.current_region = 'EMEA';
SELECT * FROM sales_data; -- Automatically returns EMEA rows only
```

B. Power BI / Analysis Services (DAX Filter Expression)

```
// Applied within Role Definition in Tabular Model
[UserEmail] = USERPRINCIPALNAME()

// Organizational Hierarchy Dynamic RLS Rule:
[ManagerEmail] = USERPRINCIPALNAME() || [EmployeeEmail] = USERPRINCIPALNAME()
```

5. Key Benefits & Strategic Advantages

- **Centralized Security Governance:** Business logic enforced at database layer eliminates reliance on application code.
- **Data Leakage Prevention:** Prevents SQL injection or flawed application logic from surfacing unauthorized rows.
- **Transparent Application Layer:** Web apps, dashboards (Power BI, Tableau), and analytical pipelines execute queries normally without explicit filter predicates.
- **Multi-Tenancy Isolation:** Allows heterogeneous tenant data to safely reside within a single database and schema.

6. Performance Considerations & Risks

Critical Technical Considerations

- **Query Execution Plans:** Security predicates modify execution paths. Unindexed filter columns in policies can trigger full table scans across millions of records.
- **Side-Channel Leakage (Timing Attacks):** Custom functions inside security policies may execute before other conditions, potentially leaking data presence via execution timing or deliberate error triggers.

- **Connection Pooling Challenges:** In application servers using shared connection pools, the session context (e.g., tenant ID) must be explicitly set and cleared on every checkout to prevent cross-tenant data leaks.